

# JavaScript Objects - Complete Notes

---

## Table of Contents

1. [What is an Object?](#)
  2. [Object Keys \(Properties\)](#)
  3. [Ways to Create Objects](#)
  4. [Object Methods](#)
  5. [Object Operations](#)
  6. [Object Copy \(Shallow vs Deep\)](#)
  7. [Built-in Object Methods](#)
  8. [Code Examples](#)
- 

## What is an Object?

- An Object is a block of memory which has state(variable), behaviour(methods) and where we can store heterogenous data.
- An object is a collection of key-value pairs that can contain various data types, such as numbers, strings, arrays, functions, and other objects.
- In one object we can have multiple key value pair and it should be separated by ',' comma.
- We can access value of object using (.) Operator or square bracket [], object reference and key\_name.

## Basic Object Structure

```
const obj = {  
  key1: value1,  
  key2: value2,  
  key3: value3 // No comma after last property  
};
```

## Empty Object

```
const obj = {  
  username: "Avinash",  
  age: 21,  
  greet: function(){  
    console.log("Good morning");  
  }  
};
```

---

# Object Keys (Properties)

## Key Conversion Rules

- **Automatic String Conversion:** Object keys are automatically converted to strings by the JS engine
- **Numeric Keys:** Number keys are converted to strings and arranged in ascending order
- **Space-separated Keys:** Must be enclosed in double quotes
- **Computed Properties:** Use square brackets `[]` with variable names
- Object key (property) will be automatically converted into string by js engine.

- If keys name are in Number, js engine will convert them into string and arrange them in ascending order.
- To write space separated key names, we have to enclose key name with double quotes.
- If we want to give computed or user defined property then we have to use square brackets and variable name.

## Examples

### 1. Numeric Keys (Auto-sorted)

```
const obj = {  
  4: "avinash",  
  1: "ranjan",  
  6: "karan",  
  3: "Abhimanyu",  
  9: "Raj"  
};  
console.log(obj); // Keys will be sorted: 1, 3, 4, 6, 9
```

### 2. Space-separated Keys

```
const obj = {  
  "first name": "John",  
  "last name": "Doe"  
};
```

### 3. Computed Properties

```
const key1 = "username";  
const obj = {  
  [key1]: "Avinash", // Dynamic key  
  age: 23  
};
```

## 4. Shorthand Property Names

```
let phone = 8800425635;
let obj = {
  phone // Same as phone: phone
};
```

# Ways to Create Objects

## 1. Object Literal (Curly Braces)

```
// Empty object
let obj = {};

// Object with properties
let obj = {
  name: "Avinash",
  age: 16
};
```

## 2. Object Constructor

```
// Empty object
let obj = new Object();

// Object with initial values
let obj = new Object({ name: "Avinash" });

// With primitive values
let obj2 = new Object(12); // Number object
let obj3 = new Object('av'); // String object
```

### 3. Constructor Function

```
function CreateObject(username, age, city) {  
    this.username = username;  
    this.age = age;  
    this.city = city;  
}  
  
const u1 = new CreateObject("Avinash", 22, "Muzaffarpur");
```

#### Key Points about Constructor Functions:

- Function name should start with capital letter
- `new` keyword creates a new object and assigns its reference to `this`
- By default, constructor functions return the newly created object

### 4. ES6 Classes

```
class Car {  
    constructor(name, price, color) {  
        this.carname = name;  
        this.price = price;  
        this.color = color;  
    }  
}  
  
const c1 = new Car("Rolls Royce", 20000000, "beige");  
console.log(c1);
```

---

## Object Methods

- In javascript, Object methods are functions that are attached to the object, and can be called on that object reference.
- To call a function, we use square brackets instead dot operator.

***Here, speak is a variable which holds the function reference.***

```
let obj1 = { name: "chombi",  
  age: 16,  
  speak: function () {  
    console.log('i can speak');  
  } }  
console.log(obj1["speak"]());  
//i can speak
```

- Object methods are functions stored as object properties.

## Basic Method Example

```
const user = {  
  username: "Avinash",  
  desg: "Mern developer",  
  intro: function() {  
    console.log(`I am ${this.username} working as a ${this.desg}`);  
  }  
};  
  
user.intro(); // Call the method
```

## Method with Return Value

```
const obj = {  
  username: "Avinash",  
  age: 21,  
  greet: function() {  
    console.log("Good morning");  
    return 10;  
  }  
};  
  
const x = obj.greet(); // x = 10
```

## Arrow Functions in Methods

```
const user = {
  username: "Avinash",
  desg: "Mern developer",
  bag: "10cr",
  intro: function() {
    console.log(`I am ${this.username} working as a ${this.desg}`);

    const inner = () => {
      console.log("I am inner function");
      console.log(`I have ${this.bag}`); // Arrow function inherits
      'this'
    };
    return inner;
  }
};
const x = user.intro();
x(); // Call the inner function
```

## Object Operations

### Accessing Object Properties

```
const obj = { username: "Avinash", age: 24 };

// Dot notation
console.log(obj.username);

// Bracket notation (recommended for dynamic access)
console.log(obj["age"]);
```

### Adding Properties

```
const obj = { username: "Avinash", age: 24 };

obj.city = "Noida"; // Dot notation
obj["state"] = "Uttar Pradesh"; // Bracket notation
```

## Checking if Property Exists

```
console.log("username" in obj); // true
console.log("job" in obj);      // false
```

## Updating Properties

```
obj.age = 25; // Update existing property
```

## Deleting Properties

```
delete obj.state; // Remove property
```

# Object Copy (Shallow vs Deep)

## 1. Shallow Copy

- The copy of object that is directly connected with original object is called as shallow object.
- Here, we store reference of original object in a new variable, now new variable starts pointing to same memory block.
- So if we make any changes in copy, it will be reflected to original object because both variables are pointing to same memory block.

```
const jack = {
  cake: "ice-cream", food: "Pizza" };
const oggy = jack; // Shallow copy (reference copy)
delete oggy.cake;
console.log("Jack:", jack);      // cake property is deleted from jack too!
console.log("Oggy:", oggy);
```



## 2. Deep Copy Methods

- The copy in which original object is not connected with its copy, is called as Deep copy.
- Here, we create separate empty object and after that we copy keyvalue pair of original object into new empty object.
- Now, if we make any changes in copy, it will not be reflected to original object because we have create separate memory blocks.

### Method 1: Spread Operator (One Level Deep)

```
const jack = { cake: "ice-cream", food: "Pizza" };  
const oggy = { ...jack }; // Creates new object
```

### Method 2: structuredClone() (True Deep Copy)

```
const jack = {  
  cake: "ice-cream",  
  info: {  
    bike: "David Putra",  
    engine: "3000 cc"  
  }  
};  
  
const oggy = structuredClone(jack); // Deep copy including nested objects  
oggy.info.bike = "Bullet";  
  
console.log("Jack:", jack); // Original unchanged  
console.log("Oggy:", oggy); // Only copy is modified
```

# Built-in Object Methods

## 1. Object.keys()

Returns an array of object's property names.

```
const obj = { username: "Avinash", age: 24, city: "Noida" };
const keys = Object.keys(obj);
console.log(keys); // ["username", "age", "city"]
```

## 2. Object.values()

Returns an array of object's property values.

```
console.log(Object.values(obj)); // ["Avinash", 24, "Noida"]
```

## 3. Object.entries()

Returns an array of key-value pairs as arrays.

```
console.log(Object.entries(obj));
// [["username", "Avinash"], ["age", 24], ["city", "Noida"]]
```

## 4. Object.fromEntries()

Creates object from key-value pair arrays.

```
const entries = [["name", "John"], ["age", 30]];
const newObj = Object.fromEntries(entries);
console.log(newObj); // { name: "John", age: 30 }
```

## 5. Object.assign()

Copies properties from source objects to target object.

```
const target = { a: 1 };  
const source = { b: 2, c: 3 };  
Object.assign(target, source);  
console.log(target); // { a: 1, b: 2, c: 3 }
```

## 6. Object.freeze()

Makes object immutable.

```
const obj = { name: "John" };  
Object.freeze(obj);  
obj.name = "Jane"; // Won't work
```

## 7. Object.isFrozen()

Checks if object is frozen.

```
console.log(Object.isFrozen(obj)); // true
```

## 8. Object.hasOwn()

Checks if object has specific property.

```
console.log(Object.hasOwn(obj, 'name')); // true
```

## 9. Object.setPrototypeOf() & Object.getPrototypeOf()

Set and get object's prototype.

```
const proto = { type: "animal" };  
const dog = { name: "Buddy" };  
  
Object.setPrototypeOf(dog, proto);  
console.log(Object.getPrototypeOf(dog)); // { type: "animal" }
```

---

# Code Examples

## Complete Working Example

```
// Create object using different methods
const user1 = {
  username: "Avinash",
  age: 24,
  city: "Noida",
  greet: function() {
    return `Hello, I'm ${this.username}`;
  }
};

// Constructor function
function User(username, age, city) {
  this.username = username;
  this.age = age;
  this.city = city;
  this.greet = function() {
    return `Hello, I'm ${this.username}`;
  };
}

const user2 = new User("Ravi", 25, "Delhi");

// Class-based approach
class UserClass {
  constructor(username, age, city) {
    this.username = username;
    this.age = age;
    this.city = city;
  }

  greet() {
    return `Hello, I'm ${this.username}`;
  }
}

const user3 = new UserClass("Priya", 23, "Mumbai");

// Testing all approaches
console.log(user1.greet());
console.log(user2.greet());
console.log(user3.greet());
```

## Key Takeaways

1. **Objects are reference types** - copying creates references, not new objects
  2. **Property keys are always strings** - numbers get converted automatically
  3. **Use bracket notation** for dynamic property access
  4. **Methods can access object properties** using `this` keyword
  5. **Choose appropriate copy method** - shallow vs deep based on your needs
  6. **Built-in Object methods** provide powerful utilities for object manipulation
  7. **Multiple creation patterns** exist - choose based on your use case
-